LexAegis AI — Backend

Production-style **Agentic Legal RAG** backend: FastAPI gateway → safety layer → hybrid retrieval → an 8-agent LangGraph workflow that returns **grounded, cited, confidence-scored** legal answers.

Built and verified through **Phase 3**. Phase 4 (observability, evaluation, frontend, Docker) is layered on next.

---

1. Quick start

```
# From the repo root
python -m venv .venv
# Windows:  .venv\Scripts\activate      |  *nix: source .venv/bin/activate

# Light deps run the WHOLE pipeline locally with no model downloads / services:
pip install -r backend/requirements-phase1.txt
pip install numpy rank-bm25 langgraph langchain-core

cp .env.example backend/.env            # then set SUPABASE_JWT_SECRET

cd backend
uvicorn app.main:app --reload           # http://localhost:8000/docs
```

> The `.env.example` ships with **light backends** selected (`EMBEDDING_BACKEND=hashing`, `RETRIEVAL_VECTOR_STORE=memory`, `RETRIEVAL_RERANKER_BACKEND=lexical`, `SAFETY_PII_BACKEND=regex`, `SAFETY_INPUT_GUARD_BACKEND=heuristic`, `AGENT_ORCHESTRATOR=langgraph`). Flip each to its production value (`bge` / `chroma` / `bge` / `presidio` / `llama_guard`) once the heavy deps and services are installed — **no code changes required**.

Smoke test in 30 seconds (no auth needed)

```
curl "http://localhost:8000/api/v1/health"
curl "http://localhost:8000/api/v1/ping?msg=hello"      # <-- test route
```

---

2. Request flow (how a `/chat` call travels the codebase)

```
HTTP POST /api/v1/chat
  |
  ▼  app/main.py            middleware: RequestContext (req-id) → CORS → Tenant
  ▼  app/api/v1/routes/chat.py
       deps.enforce_rate_limit  → app/services/rate_limiter.py   (per-user + per-
tenant)
       deps.get_current_tenant  → app/auth/supabase.py            (verify JWT →
Principal)
```

```
  ▼  app/services/chat_service.py
  ▼  app/agents/graph.py     LegalAgentWorkflow  (LangGraph StateGraph)
       guard               app/agents/guard.py            input safety + query PII
mask
        └─ blocked? ──────► refuse, END
       query_understanding app/agents/query_understanding.py  intent / task /
entities
       planner             app/agents/planner.py          workflow + retrieval
strategy
       retrieval           app/agents/retrieval_agent.py  →
app/retrieval/pipeline.py
                             dense → sparse → RRF → compress → rerank → top-K
       reasoning           app/agents/reasoning.py        answer from context only
       citation            app/agents/citation.py         [S1] tags → source refs
       groundedness        app/agents/groundedness.py     →
app/safety/output_safety.py
       confidence          app/agents/confidence.py       0–1 explainable score
       output_safety       app/agents/output_safety_agent.py  final release gate
  ▼  ChatResponse  (answer, intent, citations, confidence + breakdown,
groundedness, trace)
```

---

3. Directory map

```
backend/app/
  main.py                     # App factory: middleware, routers, exception
handlers
  core/
    config.py                 # Pydantic-Settings — single source of truth (all
phases)
    logging.py                # Structured logs + per-request correlation id
    exceptions.py             # Error hierarchy + consistent {"error": {...}}
envelopes
  auth/
    supabase.py               # Supabase JWT verify (HS256 secret / RS256 JWKS)
    models.py                 # Principal
  middleware/
    request_context.py        # X-Request-ID + access logging
    tenant.py                 # Tenant resolution
  services/
    rate_limiter.py           # Token-bucket limiter (memory; Redis-ready
Protocol)
    chat_service.py           # Bridges HTTP ⇔ agent workflow
    document_registry.py      # In-memory catalog powering the Document Explorer
  api/
    deps.py                   # DI: current principal, tenant, rate-limit guard
    router.py                 # Router aggregator
    v1/routes/
      health.py               # GET /health, /ready
      ping.py                 # GET /ping            <-- diagnostic test route
      auth.py                 # GET /auth/whoami
```

```
      documents.py            # POST /documents/upload, GET /documents
      chat.py                 # POST /chat
  llm/
    base.py / ollama_client.py / provider.py   # LLM abstraction (Qwen3 + Llama3.1
fallback)
  ingestion/
    loaders.py                # PDF / DOCX / TXT (page-preserving)
    chunking.py               # Section/clause/heading-aware chunker
    pipeline.py               # load → PII-mask → chunk → embed → index
    models.py                 # RawDocument, Chunk, ChunkMetadata
  retrieval/
    embeddings.py             # BGE (prod) + Hashing (light)
    vector_store.py           # Chroma (prod) + InMemory (light)
    sparse.py                 # Per-tenant BM25
    fusion.py                 # Reciprocal Rank Fusion
    compression.py            # Near-duplicate removal
    reranker.py               # BGE (prod) + Lexical (light)
    pipeline.py               # HybridRetriever (read + write paths)
  safety/
    pii.py                    # Presidio (prod) + Regex (light) + mask_text
    input_safety.py           # LlamaGuard (prod) + Heuristic (light)
    output_safety.py          # Groundedness + citation + PII-leak gate
  agents/
    state.py                  # AgentState threaded through the graph
    guard.py · query_understanding.py · planner.py · retrieval_agent.py
    reasoning.py · citation.py · groundedness.py · confidence.py ·
output_safety_agent.py
    graph.py                  # LegalAgentWorkflow (LangGraph + sequential
fallback)
tests/                        # 50 tests across config, auth, retrieval, safety,
agents, endpoints
```

---

4. API reference

All routes are under the `API_V1_PREFIX` (default `/api/v1`). Auth = Supabase **Bearer JWT**. Tenant is taken from the JWT (`app_metadata.tenant_id`); the optional `X-Tenant-ID` header must match it when tenant isolation is enforced.

Method	Path	Auth	Description
GET	`/health`	–	Liveness probe
GET	`/ready`	–	Readiness probe
GET	**`/ping`**	**–**	**Test route — echoes `?msg=` + request id + timestamp**
GET	`/auth/whoami`	✅	Returns the verified principal
POST	`/documents/upload`	✅	Multipart upload (`file`, `document_type`) → ingest
GET	`/documents`	✅	List documents ingested for the tenant

Method	Path	Auth	Description
POST	`/chat`	✅	Ask a grounded legal question

The `/ping` test route

A tiny, unauthenticated diagnostic to confirm the API is reachable and that request-id propagation works end-to-end. Defined in app/api/v1/routes/ping.py.

```
curl "http://localhost:8000/api/v1/ping?msg=hello"
```

```
{
  "pong": true,
  "echo": "hello",
  "service": "LexAegis AI",
  "version": "0.1.0",
  "request_id": "3f9c...",
  "timestamp": "2026-06-18T11:20:30.123456+00:00"
}
```

`POST /chat` example

```
TOKEN="<supabase HS256 jwt>"

# 1) upload a document
curl -X POST http://localhost:8000/api/v1/documents/upload \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@msa.txt" -F "document_type=contract"

# 2) ask a question
curl -X POST http://localhost:8000/api/v1/chat \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"query":"What does the confidentiality clause
require?","include_trace":true}'
```

Response (abridged):

```
{
  "query": "What does the confidentiality clause require?",
  "answer": "Based on the retrieved documents: ... [S1]",
  "intent": "contract_review",
  "confidence": 0.74,
  "confidence_breakdown": { "retrieval_similarity": 0.7, "reranker_score": 0.8,
"...": "..." },
  "citations": [{ "marker": "S1", "document_name": "msa.txt", "clause": "2.1",
```

```
  "page_number": 1 }],
    "groundedness": { "groundedness": 0.9, "citation_coverage": 1.0,
  "has_citations": true },
    "blocked": false
  }
```

---

5. Configuration

Every setting lives in app/core/config.py and is documented in `.env.example`. Highlights:

Variable	Light (default in .env.example)	Production
`EMBEDDING_BACKEND`	`hashing`	`bge`
`RETRIEVAL_VECTOR_STORE`	`memory`	`chroma`
`RETRIEVAL_RERANKER_BACKEND`	`lexical`	`bge`
`SAFETY_PII_BACKEND`	`regex`	`presidio`
`SAFETY_INPUT_GUARD_BACKEND`	`heuristic`	`llama_guard`
`AGENT_ORCHESTRATOR`	`langgraph`	`langgraph` (`sequential` to drop the dep)

Rate limiting, CORS, tenant isolation, Supabase, Ollama, Chroma, retrieval tuning, and safety thresholds are all env-configurable.

---

6. Tests

```
cd backend
pip install pytest pytest-asyncio numpy rank-bm25 langgraph langchain-core
pytest                  # 50 passed
```

The suite uses the **light backends** and mints local HS256 tokens, so it runs fully offline — no Ollama, ChromaDB, Presidio, or model downloads required.

File	Covers
`test_config.py`	Settings parsing, nested configs
`test_health.py` / `test_endpoints_phase3.py`	health, ping, upload, list, chat
`test_auth.py`	JWT verify, tenant isolation
`test_rate_limit.py`	token bucket, 429 + Retry-After
`test_chunking.py`	section/clause/heading chunking

File	Covers
`test_retrieval.py`	RRF, compression, hybrid search, tenant isolation
`test_safety.py`	PII masking, input guard, output validation
`test_llm.py`	provider fallback, Ollama parsing
`test_agents.py`	each agent in isolation
`test_workflow.py`	end-to-end graph (LangGraph + sequential)

---

7. Running with production backends (optional)

1. `pip install -r requirements.txt` (full stack).
2. Start **Ollama** and pull models: `ollama pull qwen3 && ollama pull llama3.1` (and `llama-guard3` for input safety).
3. Start **ChromaDB** (or use the persistent local client — the default).
4. For Presidio PII: `python -m spacy download en_core_web_lg`.
5. In `backend/.env`, switch the backend selectors to their production values (table in §5).

No source changes are needed — every heavy component sits behind a Protocol with the backend chosen by configuration.